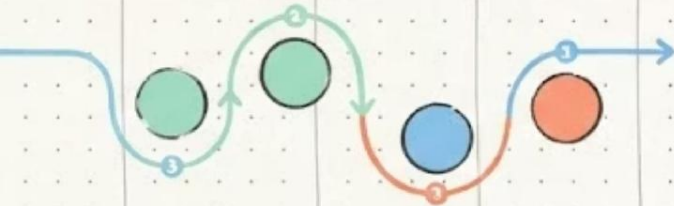


Gestão do Ciclo de Vida da Aplicação

Arquitetura de sistemas (Camadas e MVC)

Professor Romário da Silva Borges



Arquitetura em Camadas

The background features a light blue gradient with several wavy, horizontal bands of color in shades of purple, blue, and teal. These bands have a blurred, motion-like appearance. Overlaid on these bands are various snippets of code in different colors (purple, blue, green, yellow), including JavaScript, Python, and SQL, which are also blurred to match the overall aesthetic.

Arquitetura em Camadas

- Sistema é organizado em camadas, de forma hierárquica
- Camada n somente pode usar serviços da camada $n-1$
- Muito usada em redes computadores e sist. distribuídos

OSI model		
Layer	Name	Example protocols
7	Application Layer	HTTP, FTP, DNS, SNMP, Telnet
6	Presentation Layer	SSL, TLS
5	Session Layer	NetBIOS, PPTP
4	Transport Layer	TCP, UDP
3	Network Layer	IP, ARP, ICMP, IPsec
2	Data Link Layer	PPP, ATM, Ethernet
1	Physical Layer	Ethernet, USB, Bluetooth, IEEE802.11

Vantagens

- Dividir para conquistar:
 - Quebra complexidade do sistema
 - Facilita entendimento
 - Facilita troca de camadas (ex: TCP, UDP)
 - Facilita reúso de camadas (ex: várias apps usam TCP)

Variações para Sistemas de Informações

- Três camadas
- Duas camadas

Arquitetura em Três Camadas

- Comum em processos de downsizing de aplicações corporativas nas décadas de 80 e 90
- Downsizing: migração de computadores de mainframes para servidores, rodando Unix

Aplicações para Mainframes (1980-1990s)



uma única CPU

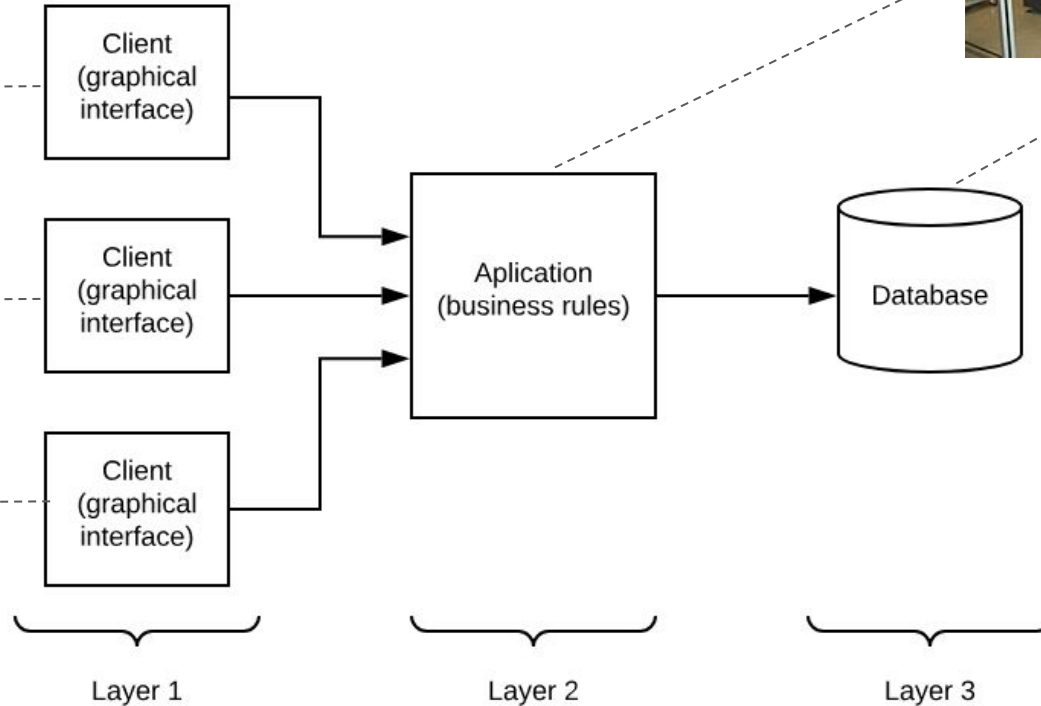


terminais burros (sem CPU);
UI baseada em texto

Arquitetura em Três Camadas



Windows



Unix

Arquitetura em Duas Camadas

- Mais simples:
 - Camada 1 (cliente): interface + lógica
 - Camada 2 (servidor de BD): bancos de dados



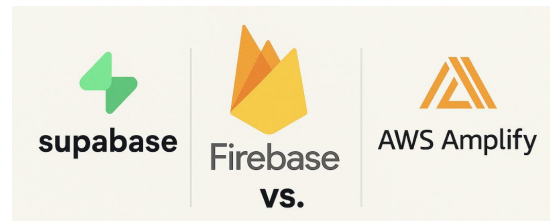
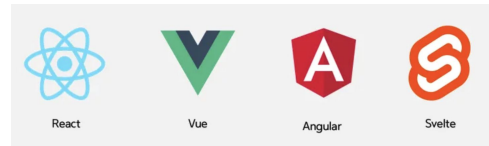
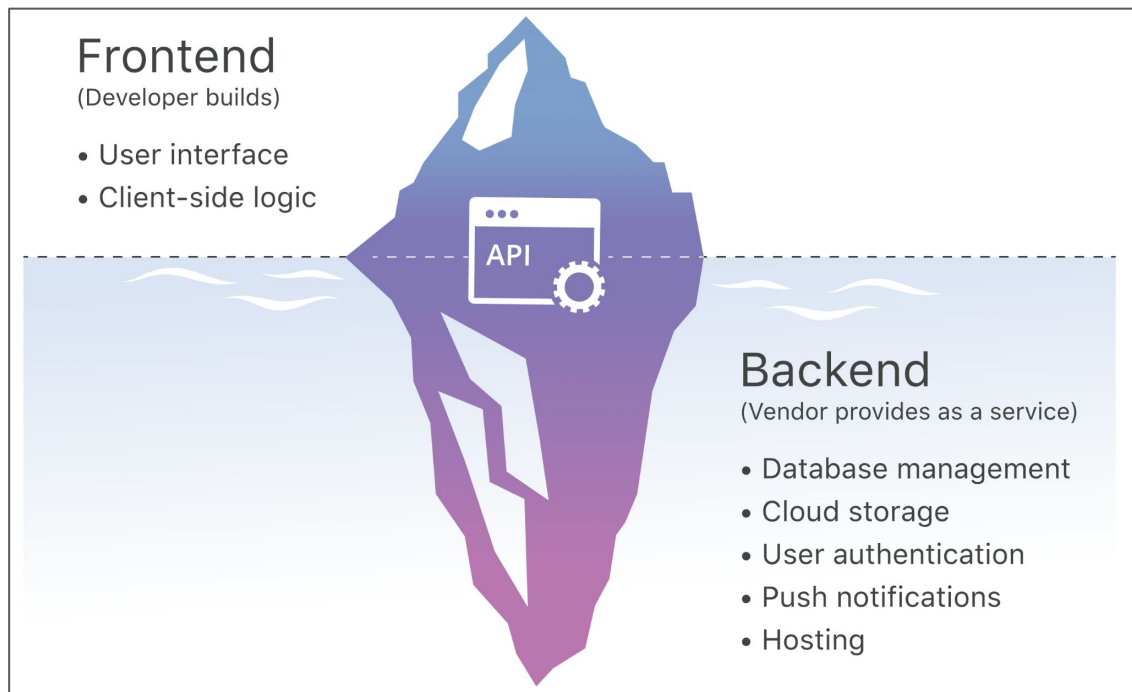
Windows



Oracle

Hoje, existe uma arquitetura parecida, chamada de Backend as a Service (BaaS)

Backend as a Service (BaaS)

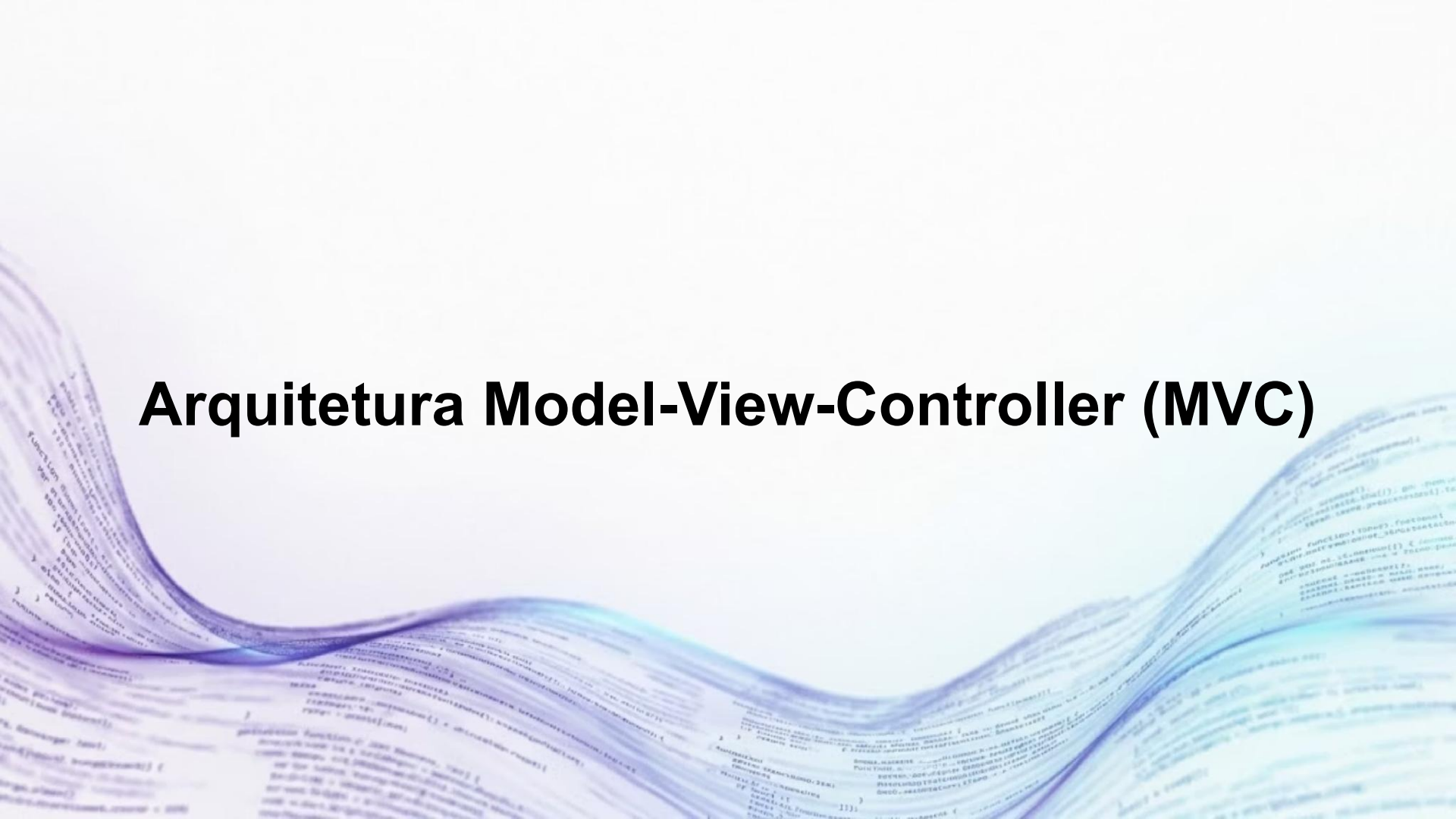


Source: <https://www.cloudflare.com/pt-br/learning/serverless/glossary/backend-as-a-service-baas/>

Exercício

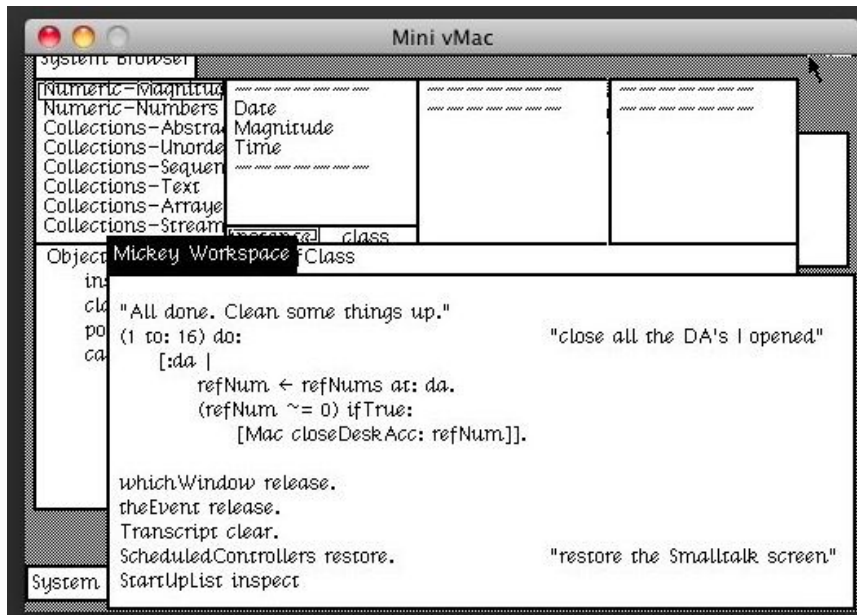
- Descreva duas desvantagens de BaaS.
- Quais as diferenças entre BaaS e Duas Camadas?

Arquitetura Model-View-Controller (MVC)

The background features a series of fluid, wavy lines in shades of blue and purple that sweep across the frame. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java, which are partially obscured by the motion of the lines.

Arquitetura MVC

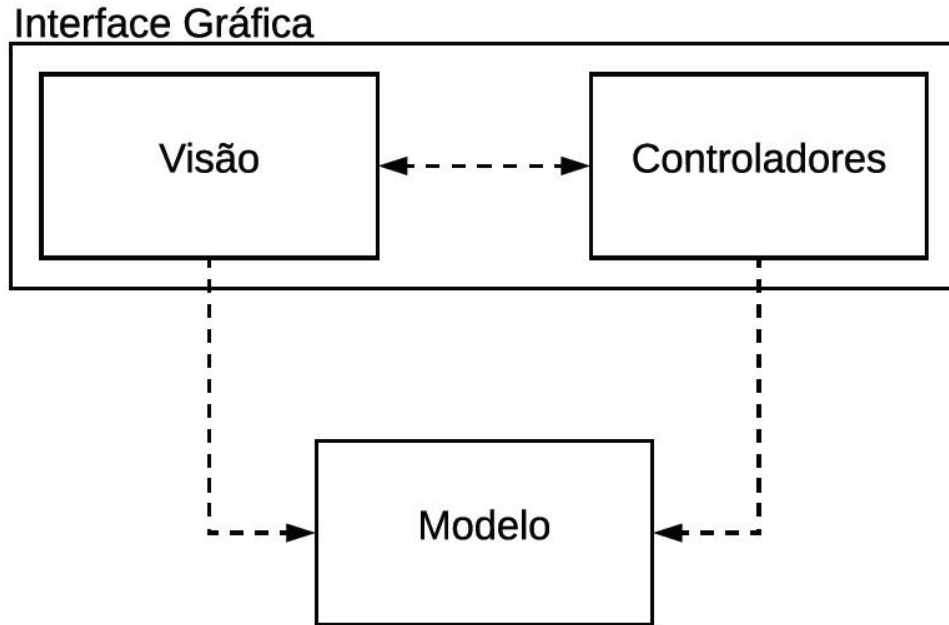
- Surgiu na década de 80, com a linguagem Smalltalk
- Proposta para implementar interfaces gráficas (GUIs)

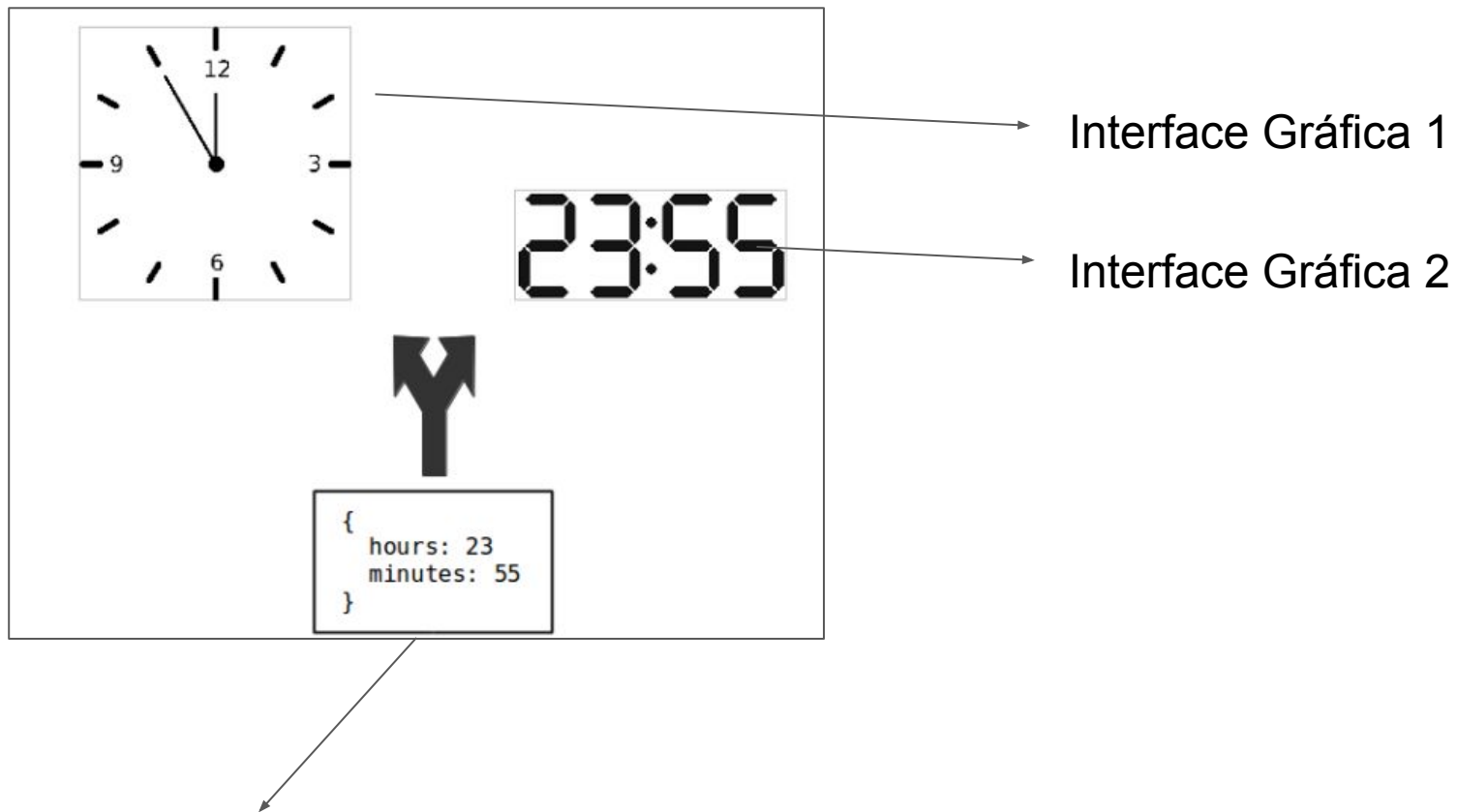


MVC

- **Visão:** classes para implementação de GUIs, como janelas, botões, menus, barras de rolagem, etc
- **Controle:** classes que tratam eventos produzidos por dispositivos de entrada, como mouse e teclado
- **Modelo:** classes com lógica e dados da aplicação

MVC = (Visão + Controladores) + Modelo
= Interface Gráfica + Modelo

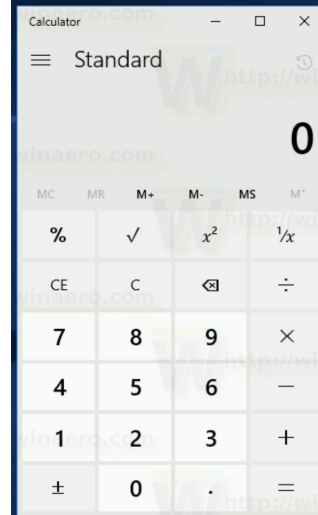
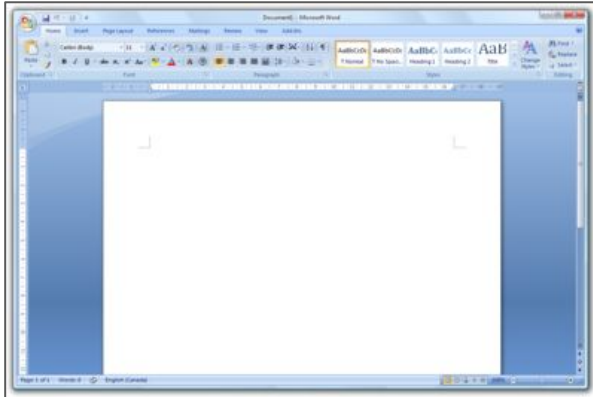




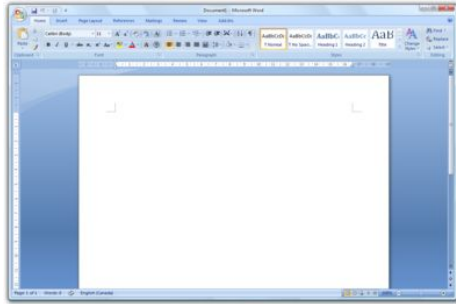
Modelo

Importante

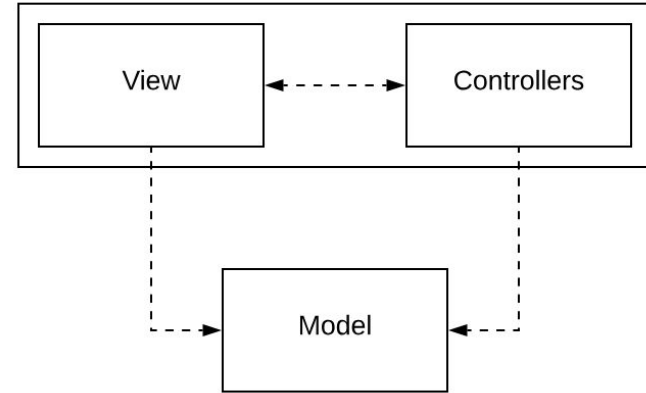
- MVC não foi pensado para aplicações distribuídas; mas para aplicações desktop monolíticas
- Exemplo: Microsoft Word, Calculadora, etc



MVC Tradicional (Smalltalk)



Graphical Interface



MVC nos dias de hoje

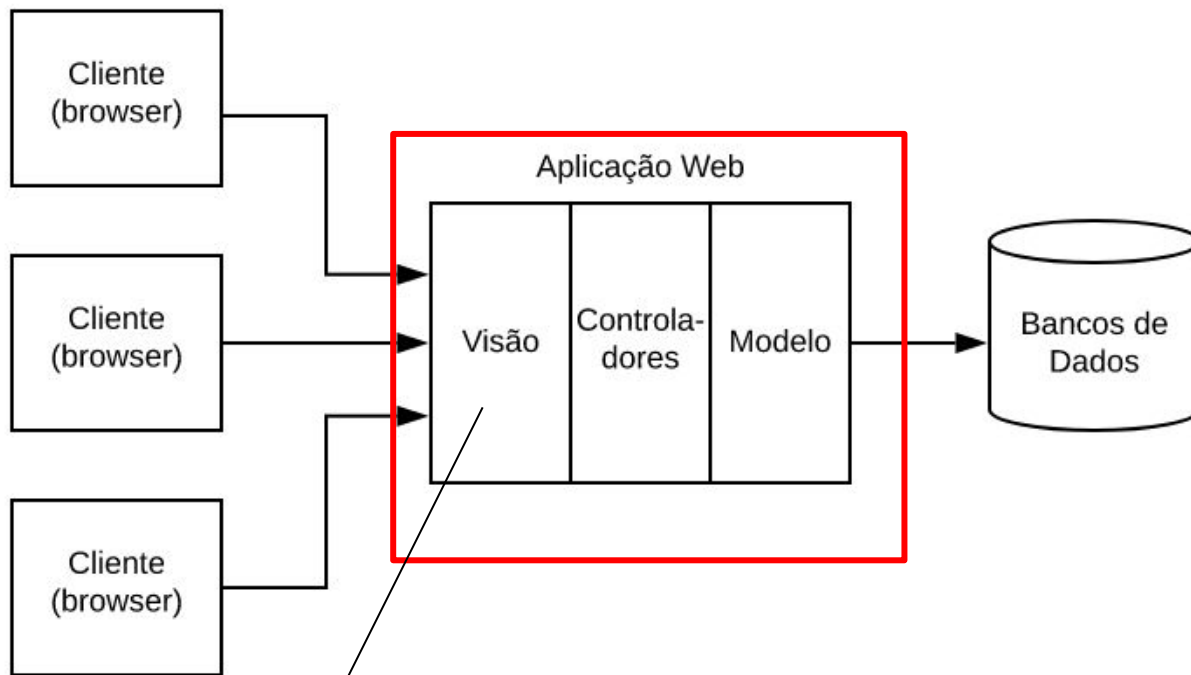
- MVC Web
- Single Page Applications

MVC Web

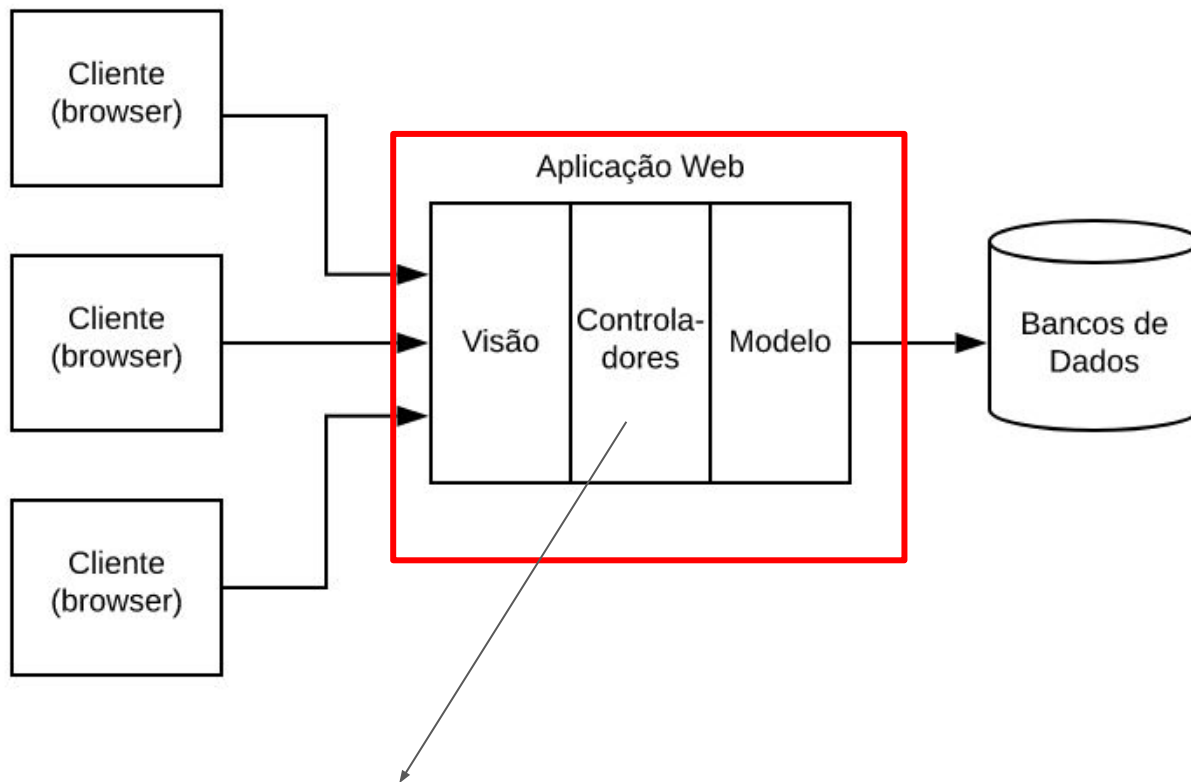
The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of motion and depth. Scattered across the waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, PHP, and Java. The code is mostly illegible due to its low opacity and the motion blur effect, but it adds a technical, digital feel to the overall composition.

MVC Web

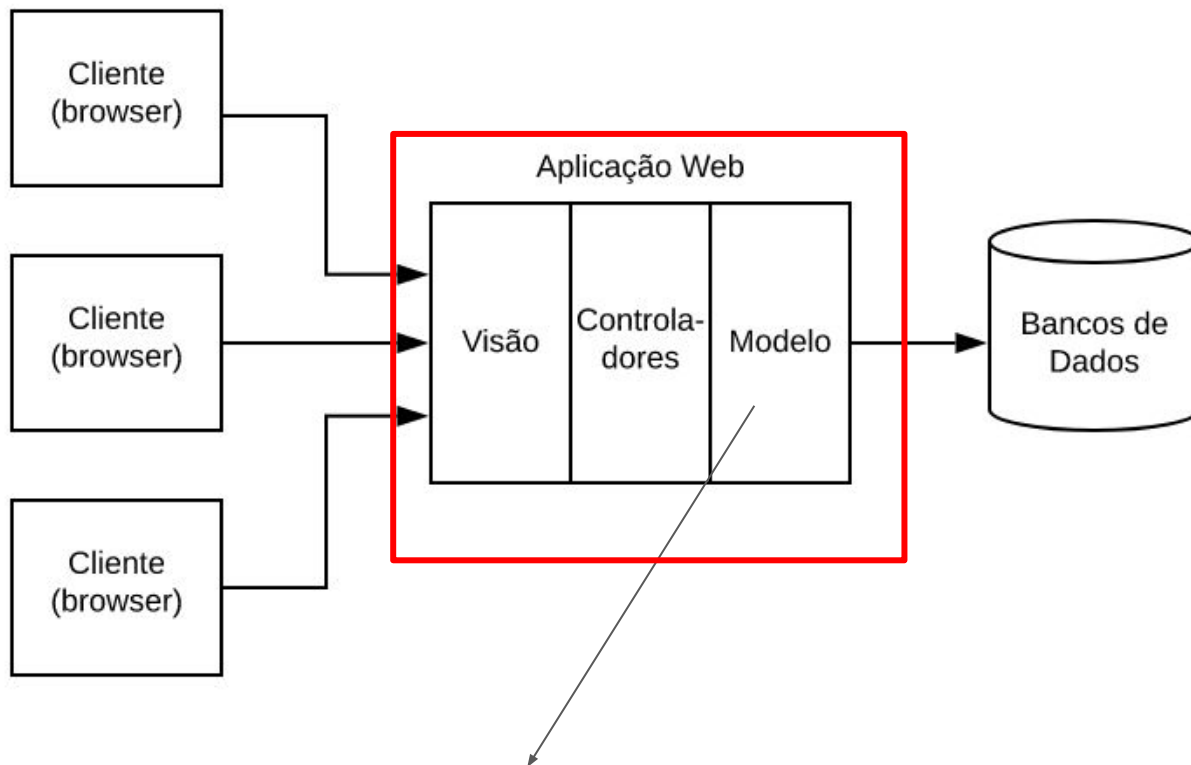
- Adaptação de MVC para Web, ou seja, sistemas distribuídos
- Usando Ruby on Rails, Django, Spring, PHP Laravel, Dotnet e etc



Páginas HTML, CSS, JavaScript
(o que o usuário vai ver)



Recebem dados de entrada e fornecem informações para páginas de saída



Lógica da aplicação (regras de negócio) e fazem a interface com o banco de dados

Exemplo muito simples de um sistema MVC Web

<https://github.com/mtov/ESM-ExemplosCodigo/tree/master/cap7/mvc>

Este exemplo não usa nenhum framework e possui uma interface Web muito simples, tudo por motivos didáticos.

No moodle há também um exemplo similar.

Controlador

```
public class ControladorPesquisaLivros {  
    ...  
    public void start() {  
        ...  
        get("/", (req, res) -> {  
            res.redirect("index.html");  
            return null;  
        });  
        ...  
    }  
}
```

Browser (index.html)

Biblioteca MVC

Pesquisa de Livros

Informe o nome do autor

Pesquisar

Nomes válidos: valente, fowler e gof

Controlador

```
public class ControladorPesquisaLivros {  
    ServicoPesquisaLivros pesq;  
    PaginaDadosLivro pagina;  
    ...  
    public void start() {  
        ...  
        get("/pesquisa", (req, res) -> {  
            String autor = req.queryParams("autor");  
            Livro livro = pesq.pesquisaPorAutor(autor) ;  
            return pagina.exibeLivro(livro.getTitulo(),  
                                     livro.getAutor(), livro.getISBN());  
        });  
        ...  
    }  
}
```

Modelo

```
public class ServicoPesquisaLivros {  
  
    public Livro pesquisaPorAutor(String autor) {  
        try(Connection con = DriverManager.getConnection(...)) {  
            String query = "select * from livros where autor = ?";  
            PreparedStatement stmt = con.prepareStatement(query);  
            stmt.setString(1, autor);  
            ResultSet rs = stmt.executeQuery();  
            String isbn = rs.getString("isbn");  
            String titulo = rs.getString("titulo");  
            return new Livro(isbn, autor, titulo);  
        } catch (SQLException e) {  
            System.out.println(e.getMessage());  
            return null;  
        }  
    }  
}
```

30 }

Controlador

```
public class ControladorPesquisaLivros {  
    ServicoPesquisaLivros pesq;  
    PaginaDadosLivro pagina;  
    ...  
    public void start() {  
        ...  
        get("/pesquisa", (req, res) -> {  
            String autor = req.queryParams("autor");  
            Livro livro = pesq.pesquisaPorAutor(autor);  
            return pagina.exibeLivro(livro.getTitulo(),  
                                livro.getAutor(), livro.getISBN());  
        });  
        ...  
    }  
}
```

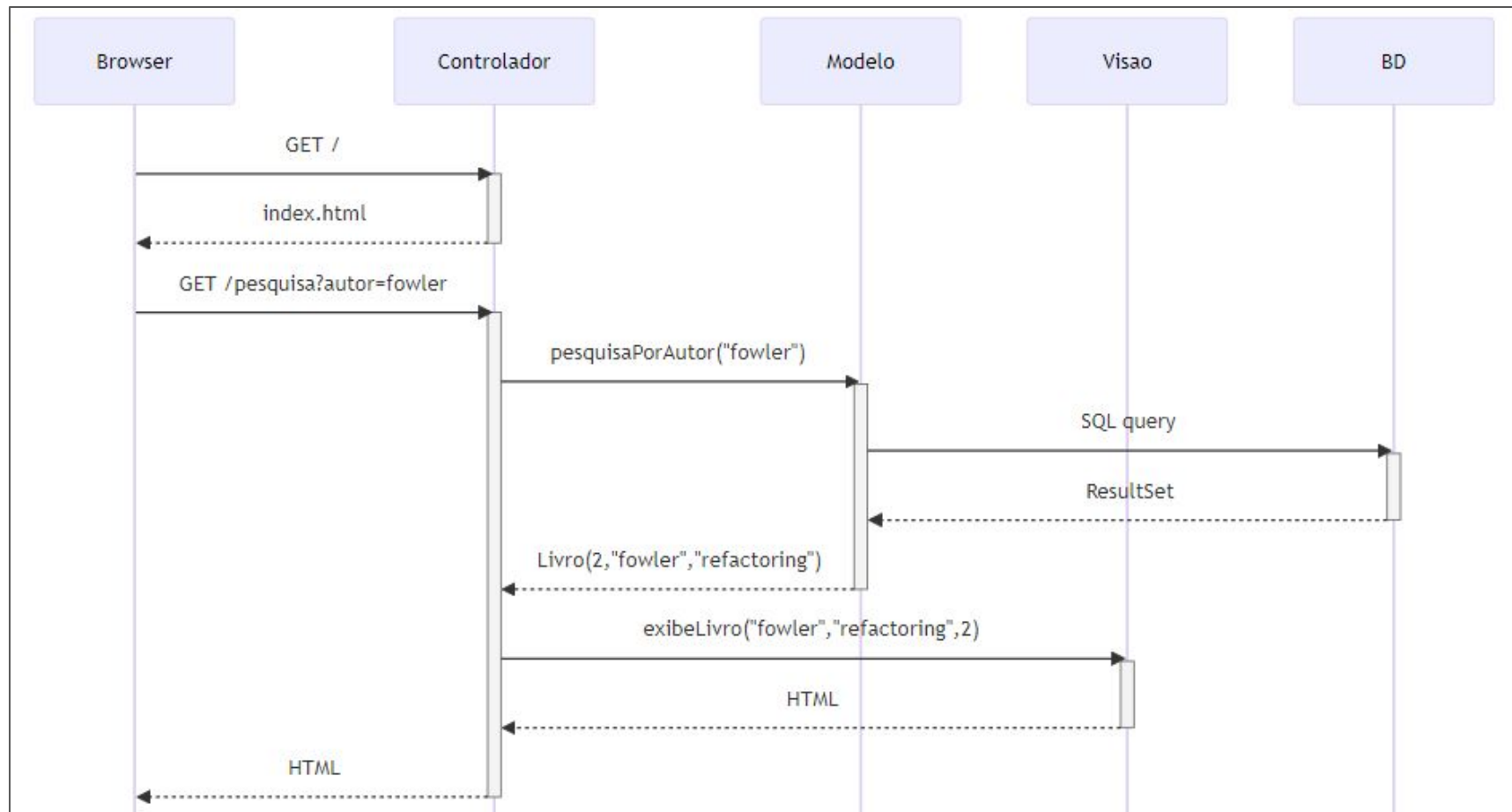
Visão

```
public class PaginaDadosLivro {  
  
    public String exibeLivro(String titulo, String autor, String isbn) {  
        String res = "<h4> Dados do Livro Pesquisado </h4>";  
        res += "<ul>";  
        res += "<li> Título: " + titulo + " </li>";  
        res += "<li> Autor: " + autor + " </li>";  
        res += "<li> ISBN: " + isbn + " </li>";  
        res += "</ul>";  
        return res;  
    }  
}
```

Browser

Dados do Livro Pesquisado

- Título: Refactoring
- Autor: fowler
- ISBN: 2



Regra de negócio um pouco mais complexa

1. Validar dados do pedido
2. Verificar disponibilidade de estoque
3. Calcular valor bruto dos itens
4. Aplicar cupom de desconto
5. Aplicar desconto por plano de fidelidade
6. Calcular frete
7. Calcular valor total do pedido
8. Validar método de pagamento
9. Autorizar pagamento
10. Atualizar estoque
11. Gerar pedido confirmado

No exemplo da pesquisa de livro, o Modelo apenas fazia uma query SQL. Porém, em sistemas maiores, o modelo pode ter que implementar regras de negócio mais complexas, como no exemplo ao lado.

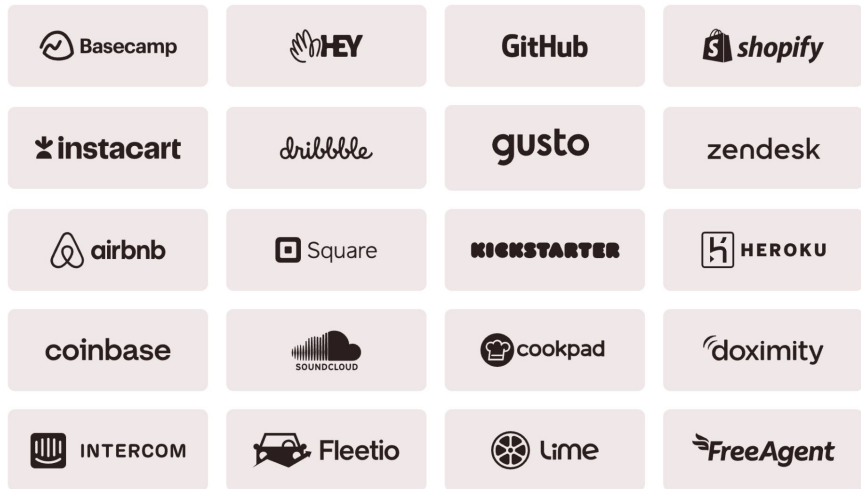
Observação

- MVC $\Rightarrow$ Aplicação Monolítica (Monolito)
- Esse monolito possui frontend e backend integrados

MVC Frameworks continuam sendo relevantes!

You're in good company.

Over the past two decades, Rails has taken countless companies to millions of users and billions in market valuations.



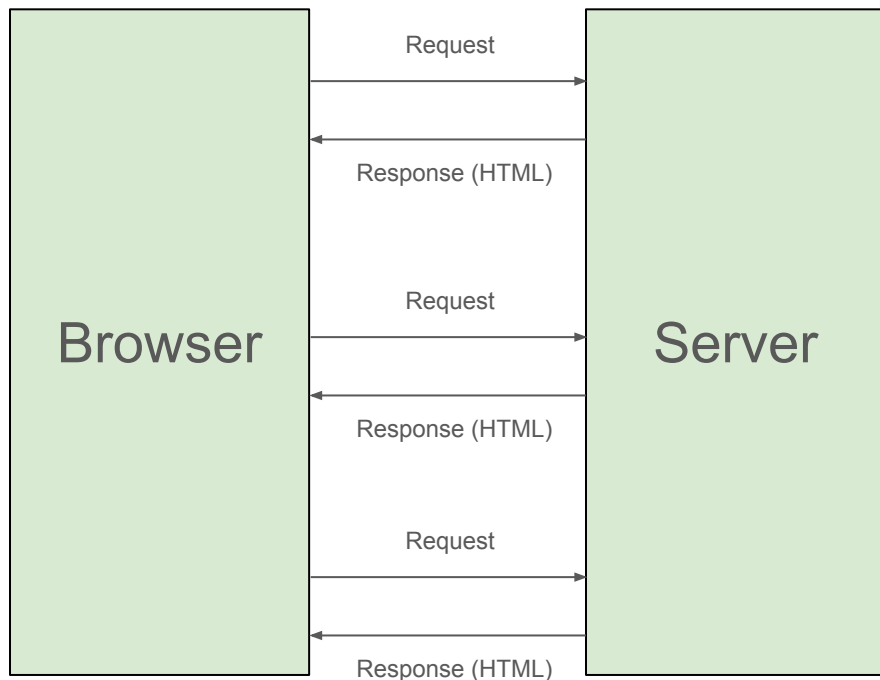
<https://rubyonrails.org> (Maio 2025)

Pergunta: Conhece alguma empresa brasileira
que usa frameworks MVC?

Single Page Applications (SPAs)

The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Faint, semi-transparent snippets of code are visible, particularly on the left and right sides, suggesting a technical or programming theme. The overall aesthetic is modern and digital.

Aplicações Web Tradicionais



Problema: interface
menos responsiva

Exemplo: Toda alteração
que faz em um formulário,
tem que clicar em “Salvar”

Single Page Applications

- Roda no browser; logo, mais independente do servidor
- "Menos burra": manipula sua interface e armazena dados
- Acessa o servidor para buscar mais dados
- Exemplo: GMail, Google Docs, Facebook, Figma, etc
- Implementadas em JavaScript, usando React, Vue, Svelte, etc



Exemplo: Aplicação Simples usando Vue.js

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Interface (Web)
HTML

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa  
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Uma Simples SPA

Temperatura: 60

Incrementa

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Modelo

Dados

Métodos

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa  
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa  
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Resumo

MVC Tradicional

(Smalltalk): aplicações gráficas, pré-Web



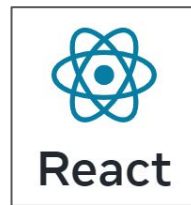
MVC Web: adaptação de MVC para Web (fullstack)



django



SPA: adaptação de MVC para uso no front-end



Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: <https://engsoftmoderna.info>.

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

